

groepswerk OOP

Technische Documentatie: Cyber-LAN Hub

Project Info

Project: Cyber-LAN Hub Groepswerk

Vak: Full Stack Development

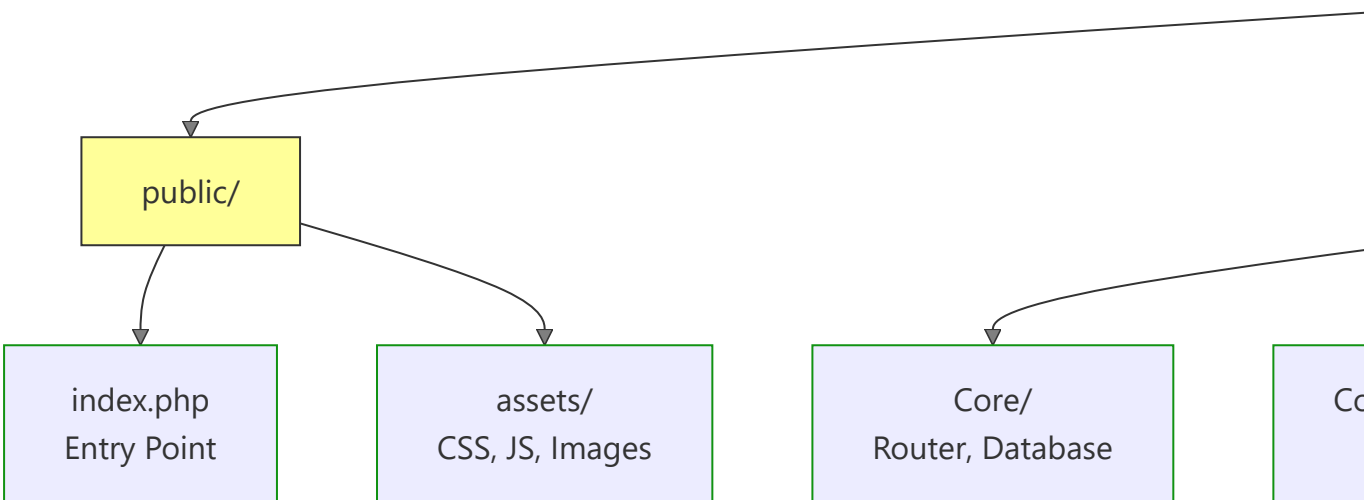
Status: Oplevering

1. Inleiding

Dit document beschrijft de technische architectuur van de **Cyber-LAN Hub** applicatie. Het doel is om inzicht te geven in hoe de applicatie "onder de motorkap" werkt, hoe de code is gestructureerd volgens het MVC-model, en hoe de verschillende componenten met elkaar samenwerken.

2. Projectstructuur

De applicatie volgt een strikte scheiding van verantwoordelijkheden. Hieronder de belangrijkste mappen en hun functie, gevisualiseerd in een boomstructuur.

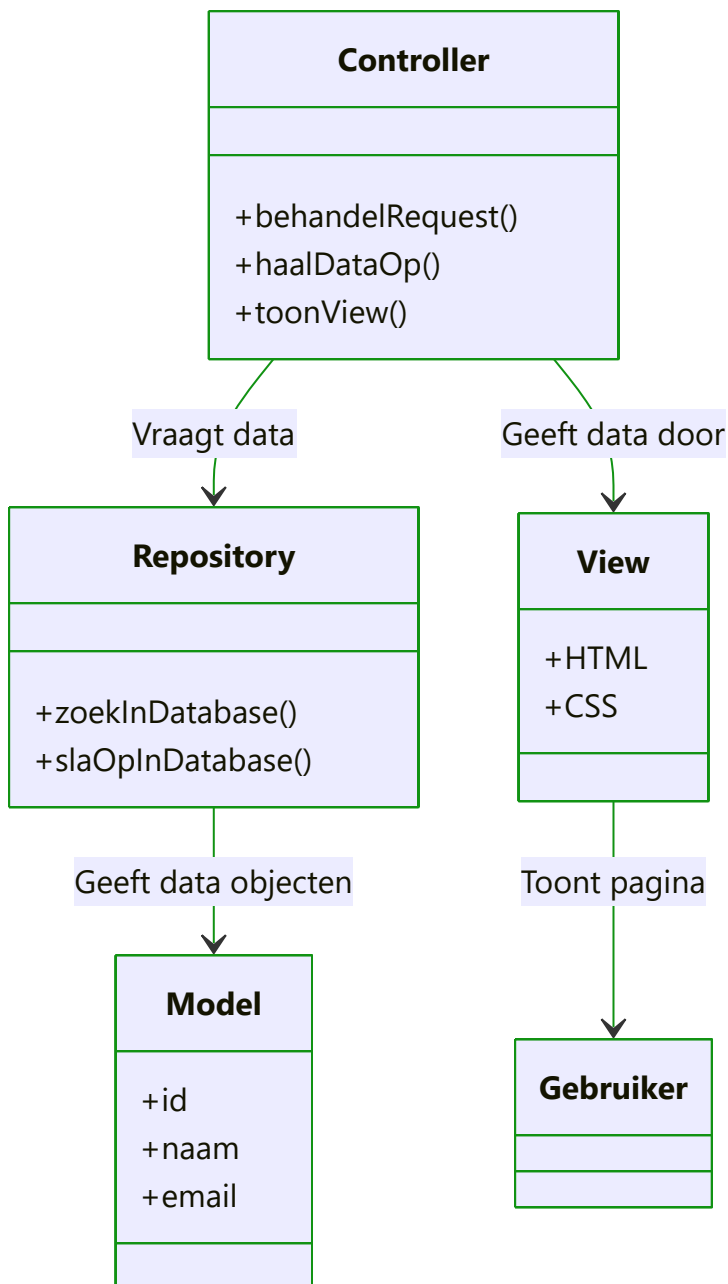


Belangrijke Mappen

- `public/` : De enige map die toegankelijk is voor de buitenwereld (webroot). Hier staat `index.php` (het startpunt).
 - `app/Core/` : Framework-achtige basiscomponenten (Router, Database, Helpers).
 - `app/Controllers/` : Behandelt de inkomende verzoeken en stuurt de logica aan.
 - `app/Repositories/` : Behandelt alle database-communicatie (scheiding van SQL en Logica).
 - `app/Views/` : De HTML templates voor de weergave.
-

3. MVC Implementatie (Model-View-Controller)

Wij hebben gekozen voor een **MVC-architectuur** met een toevoeging van het **Repository Pattern** om de code overzichtelijk en onderhoudbaar te houden.



Componenten Uitleg

1. **Repositories** (`app/Repositories/`):

- * Bevat alle SQL-logica.

De Controller praat nooit* direct met de database, maar altijd via een Repository.

- * *Voorbeeld:* `UserRepository->findById($id)`

2. **Views** (`app/Views/`):

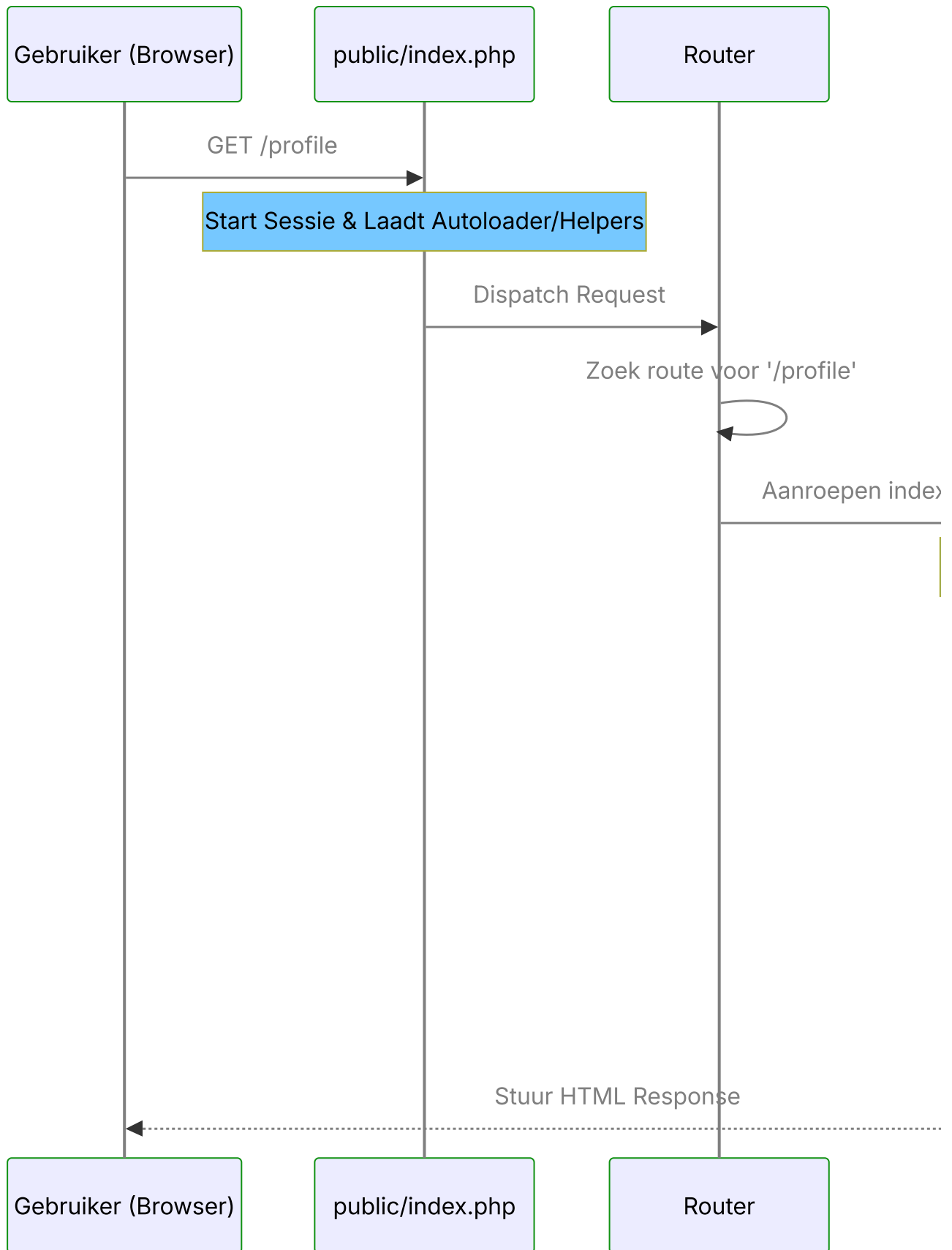
- * HTML templates die data ontvangen van de Controller.
- * Gebruikt een **Layout-systeem** (`layouts/main.php`) zodat de header/sidebar niet op elke pagina gekopieerd hoeft te worden.

3. **Controllers** (`app/Controllers/`):

- * De "Manager" van het proces.
 - * Ontvangt input, haalt data, en kiest de juiste view.
-

4. Stap-voor-Stap Flow: Request tot Response

Om te begrijpen hoe alles samenkomt, volgen we de weg die een verzoek aflegt wanneer een gebruiker naar **/profile** gaat.



Gedetailleerde Beschrijving

1. Entry Point (`public/index.php`)

- * Alle verzoeken gaan via `.htaccess` naar `index.php`.
- * Hier wordt de applicatie "opgestart" (bootstrapping).

2. Routing (`app/Core/Router.php`)

* De Router kijkt naar de URL en zoekt in `app/routes.php` welke Controller erbij hoort.

* *Match:* `/profile` → `ProfileController` .

3. Controller Actie (`ProfileController.php`)

* De `index()` methode wordt gestart.

* Beveiligingscheck: Is de gebruiker ingelogd?

* Data ophalen via de `UserRepository` .

4. Database (`app/Core/Database.php`)

* De Repository gebruikt de centrale `Database` class (Singleton) om verbinding te maken.

* Queries worden uitgevoerd met `pdo` (veilig tegen injecties).

5. View Rendering

* De `view()` helper functie combineert de specifieke pagina (`user/profile.php`) met de hoofdlayout (`layouts/main.php`).

* Het resultaat wordt teruggestuurd naar de browser.